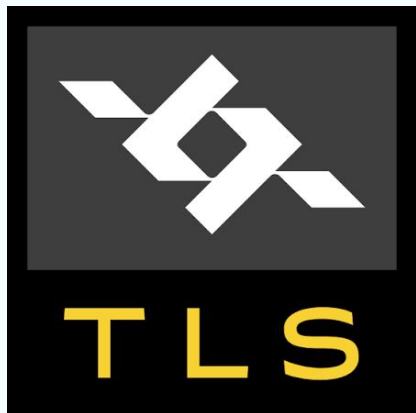


# PRACTICAL CRYPTOGRAPHY

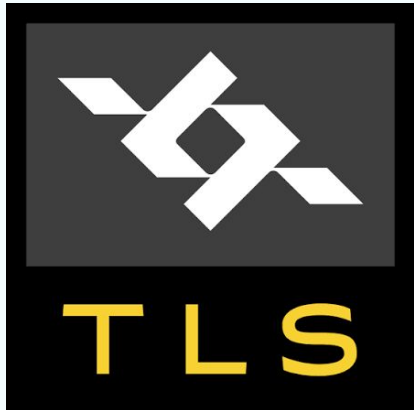
crypt@b-it 2026

Summer school on Cryptography

Sofía Celi



# LECTURE 3: A PRACTICAL PROTOCOL: TRANSPORT LAYER SECURITY -TLS- 1.3



# TLS 1.3

- For many, the *true* practical protocol: the happy case
- Encrypts the majority of the web (and beyond):
  - Almost everywhere on the Internet

Why did it get deploy?

- It is not an easy protocol to deploy, but we cared about:
  - Protecting communications from network adversaries: passwords, credit card information, PI
  - Having a protocol that did not degrade the “fastness” of the web (and beyond)
  - Solve and fix attacks of *previous versions* of the protocol

# TLS 1.3

Why did it get deploy?

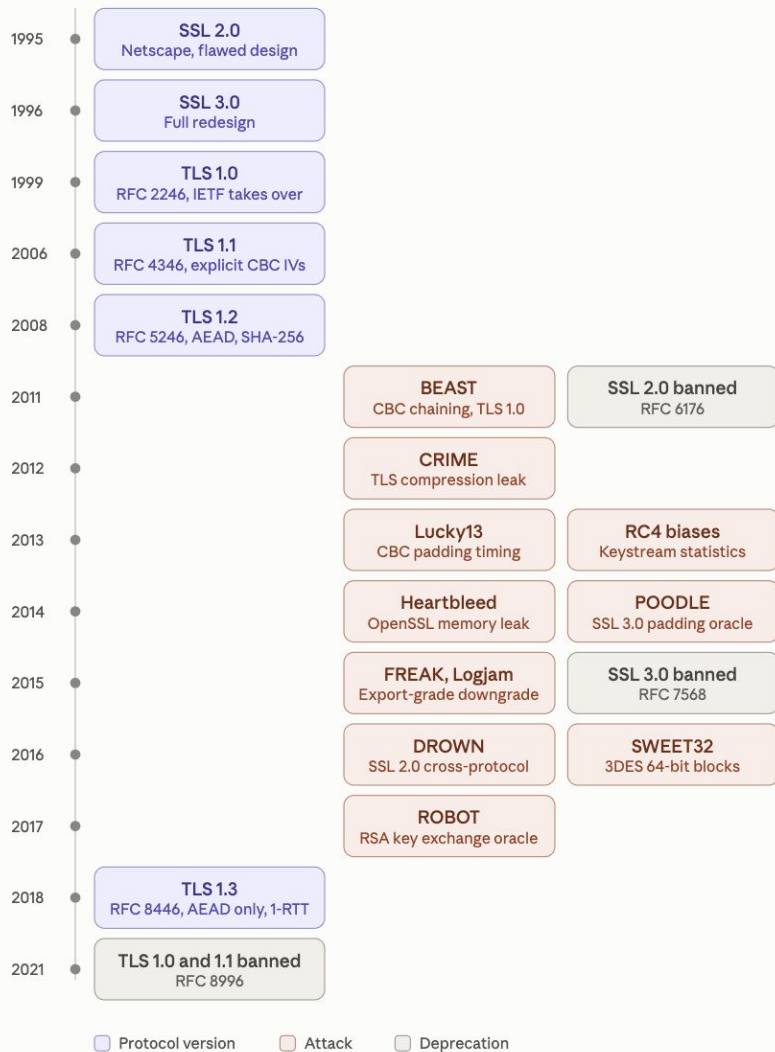
- It is not an easy protocol to deploy, but we cared about:
  - Protecting communications from network adversaries
  - Having a protocol that did not degrade the “fastness” of the web (and beyond)
  - Solve and fix attacks to previous versions of the protocol
- Cheap?
  - Not really: it costs a lot to migrate infrastructure
  - But the primitives indeed are “cheap”
- Easy to deploy?
  - Not really: requires specialised knowledge to maintain

# TLS 1.3

Why did it get design?

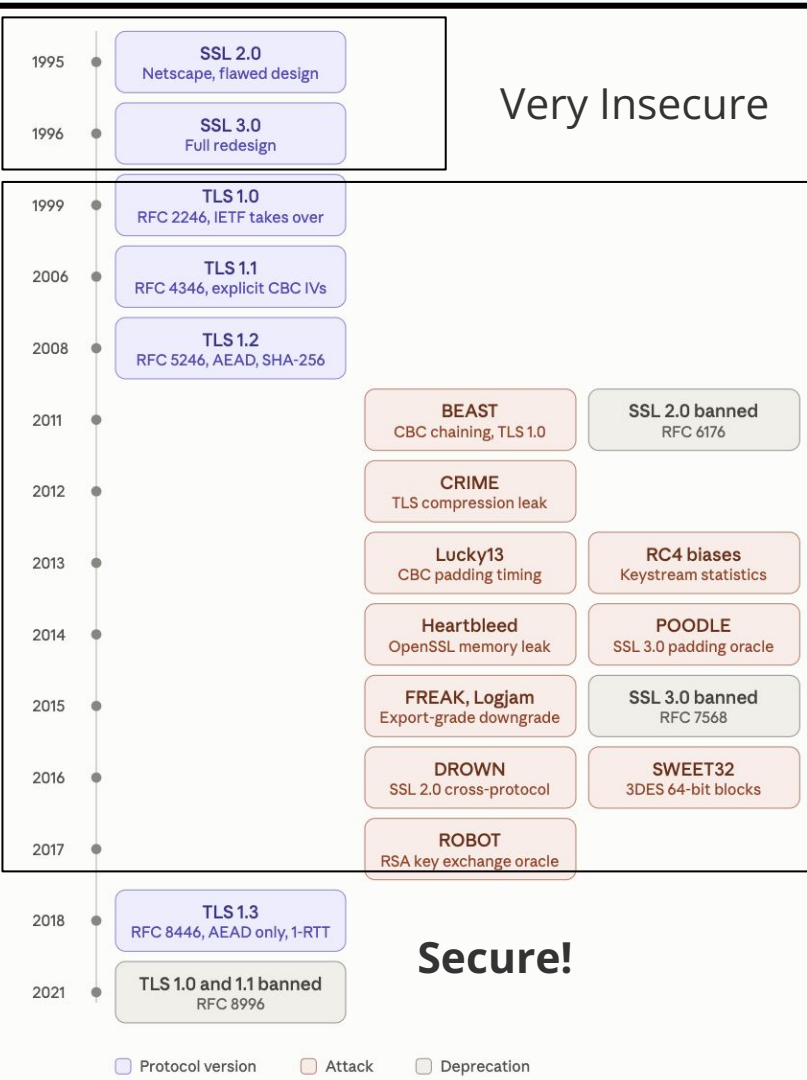
- Previous versions of TLS were insecure:
  - An explicit goal from 1.3 is to encrypt as many parts of the protocol as possible
  - It provides (*mutual*) authentication so that active attacks are not possible
  - It adds secure and fast algorithms
- Interoperability:
  - It sits on top of any “network” protocol (with caveats)
- Efficiency:
  - It introduces a 1-RTT handshake (even 0-RTT with caveats!)

# TLS 1.3



# TLS 1.3

Insecure



# TLS 1.3

*The TLS 1.3 protocol.* In reality, two protocols:

1. *Handshake protocol:*

- a. Sets up the key material for both encrypting the handshake messages as well as the record-layer messages, via the **exchange of keys** and a **“Key Schedule” routine**
- b. Authenticates parties (mandatory: the server)
- c. Provides integrity
- d. Provides forward secrecy per session

2. *Record layer protocol*

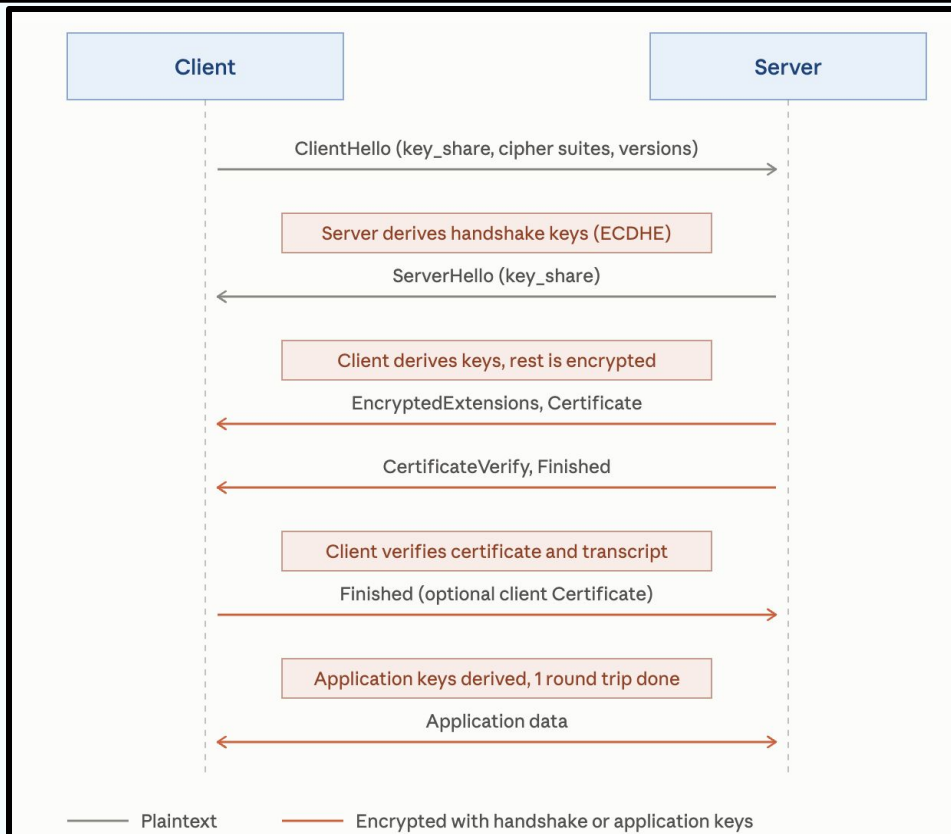
- a. Encrypts application layer data via *Authenticated Encryption with Associated Data (AEAD)* algorithms

# TLS 1.3

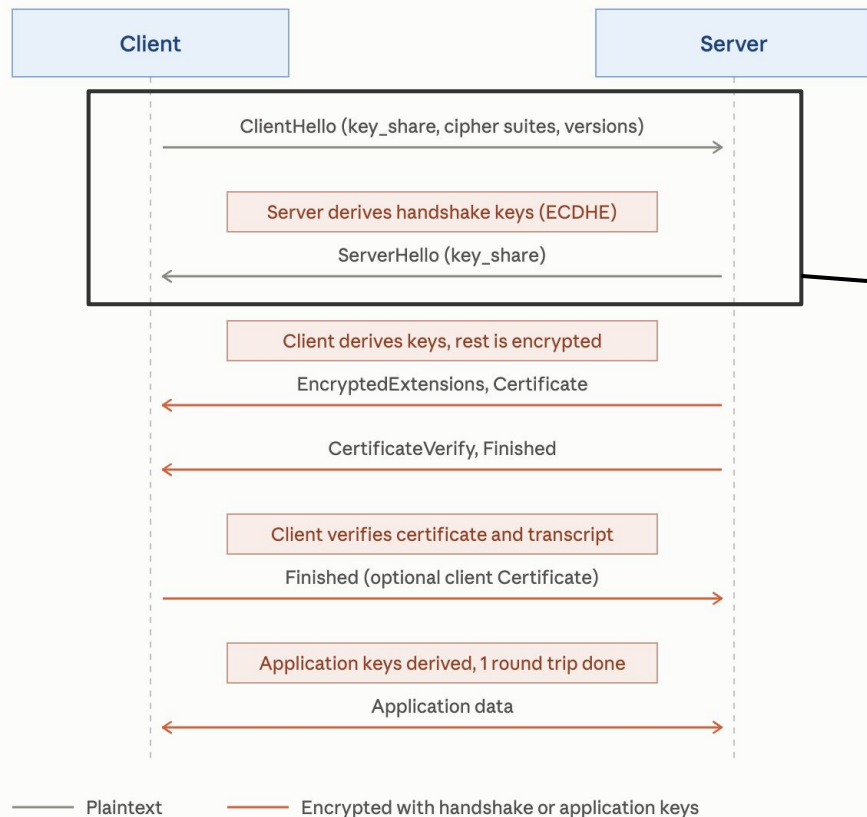
Note: it has many modes:

- 1-RTT with PSK-DHE or just PSK → aka, adding resumption
- 0-RTT → aka, adding resumption with data on the first flight
- Possibility of authentication via passwords
- Exported authenticators and post-handshake authentication

# TLS 1.3: handshake protocol

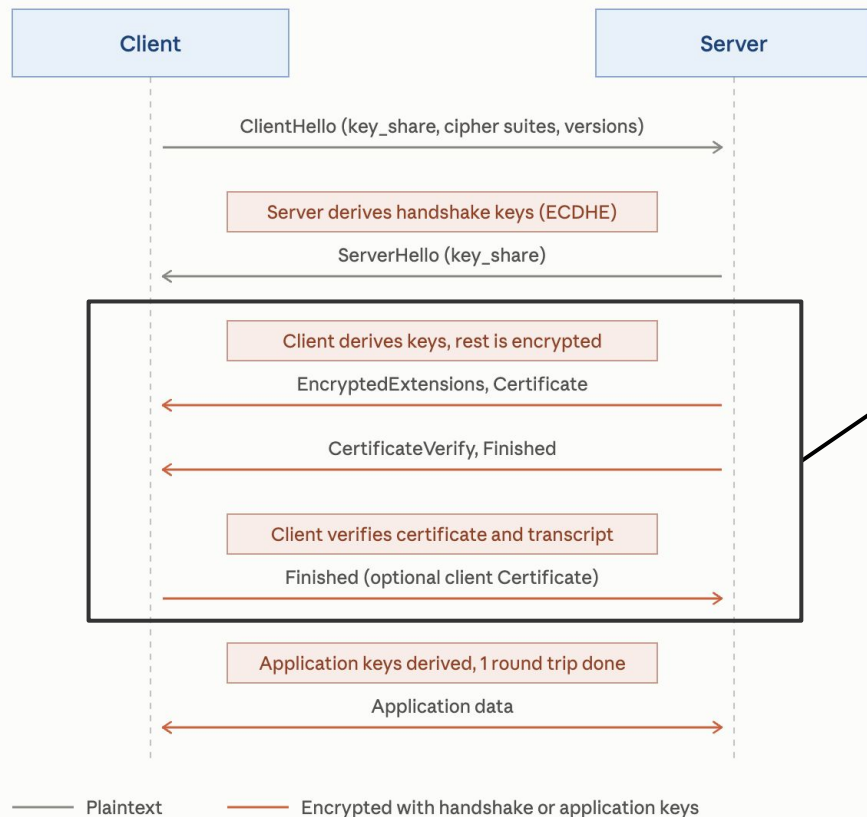


# TLS 1.3: handshake protocol



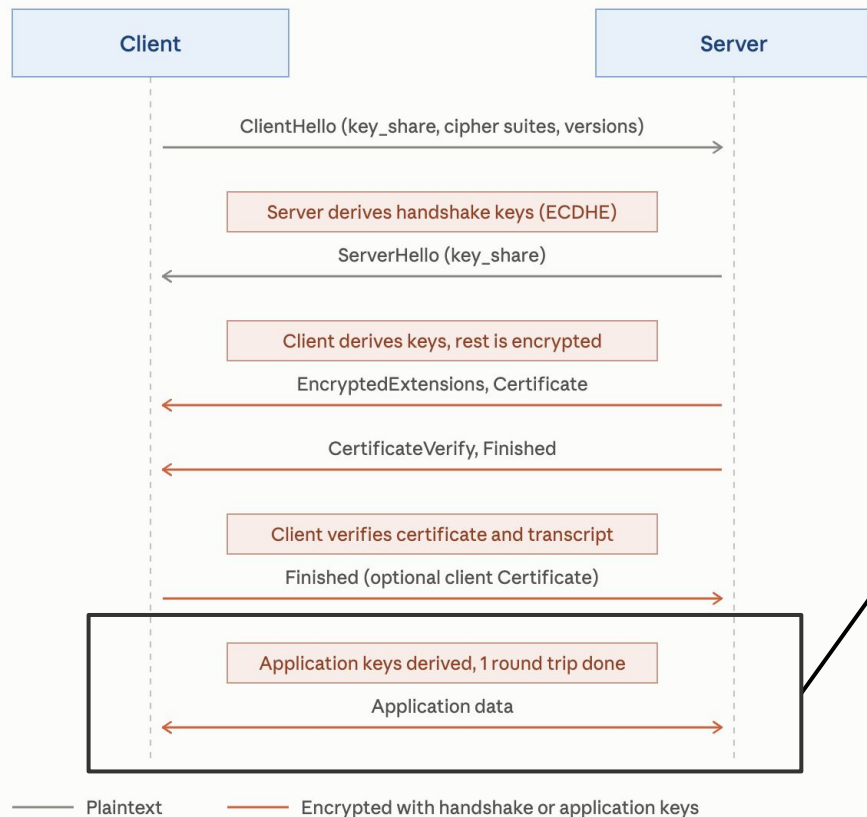
**Key Exchange,**  
which starts the  
**KeySchedule**  
protocol

# TLS 1.3: handshake protocol



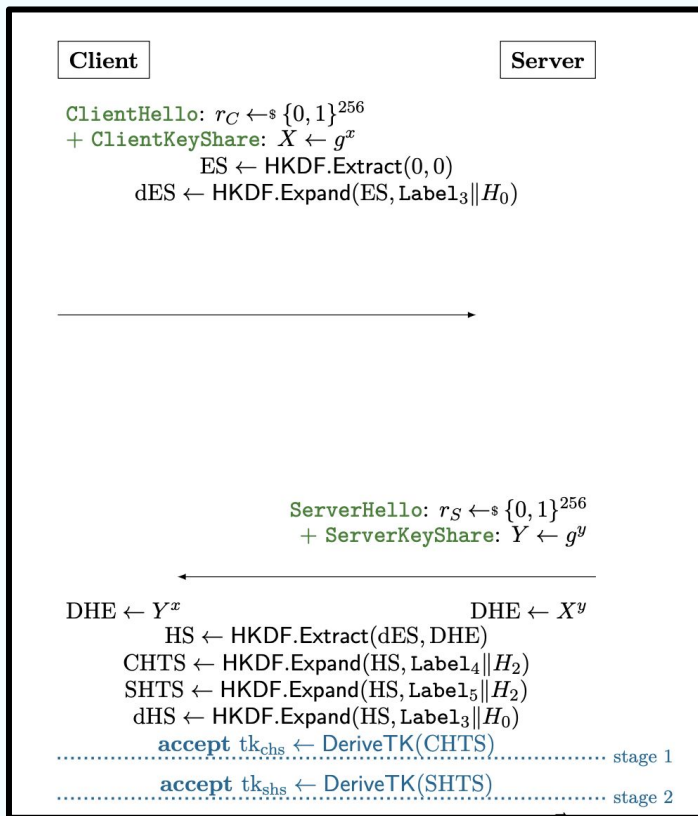
**Authentication and Integrity** messages

# TLS 1.3: handshake protocol



Record layer (not handshake,  
but here for clarity)

# TLS 1.3: handshake protocol, key exchange

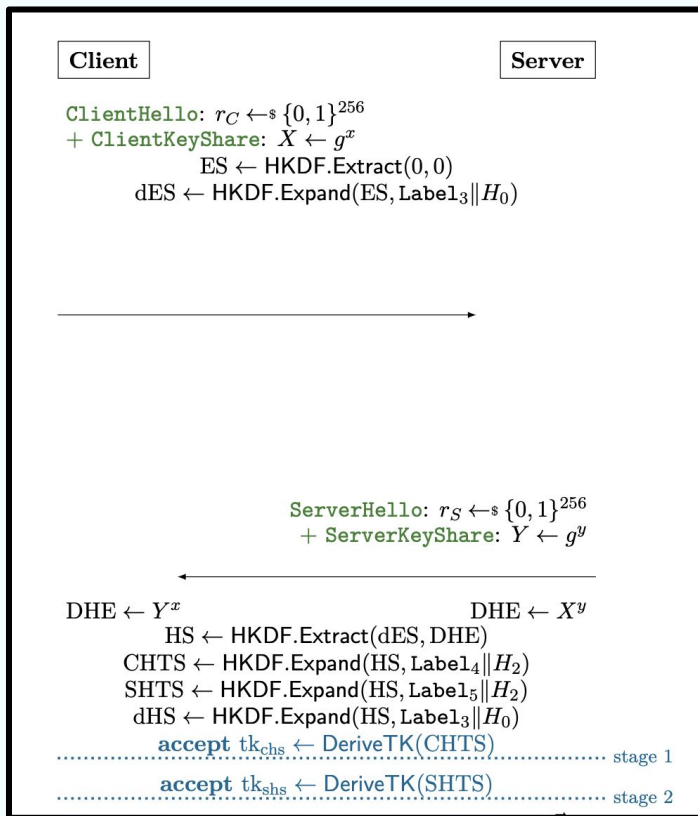


## The Key Exchange:

- Client and Server advertise their “key shares”: the public counterpart of the private value
  - Key shares are fresh (EC)DHE values
  - For replay protection, it also advertises a random value
- Client advertises many values and the Server chooses one
- Client advertises, as well, many choices for signature algorithms and encryption (AEAD) suites

Awesome diagrams from “A Cryptographic Analysis of the TLS 1.3 Handshake Protocol”,  
<https://d1kjiwibowugqa.cloudfront.net/files/research/papers/JC-DF-GS21.pdf>

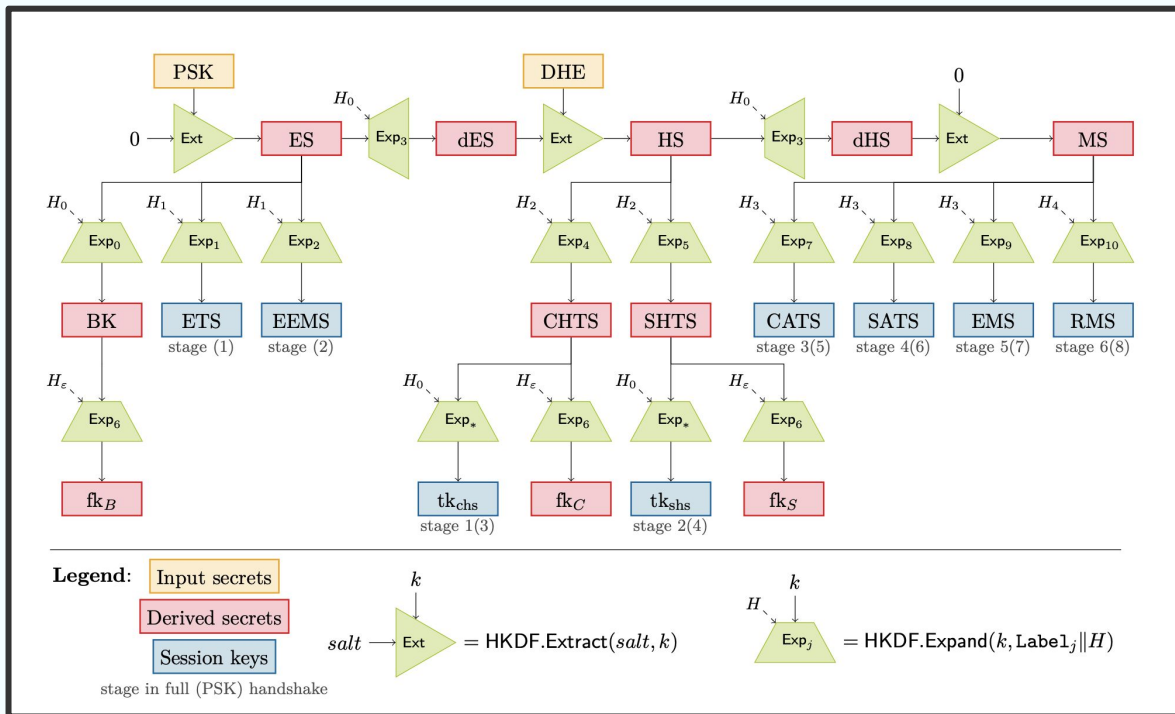
# TLS 1.3: handshake protocol, key exchange



- The client always initiates the connection by sending a `ClientHello` message: practical as usually clients initiate requests
- Server responds with `ServerHello`, and sets the “ciphersuites” to use
- From now on, the server can start the “**KeySchedule**” protocol and derive the keys used to encrypt the handshake and, later, the record layer

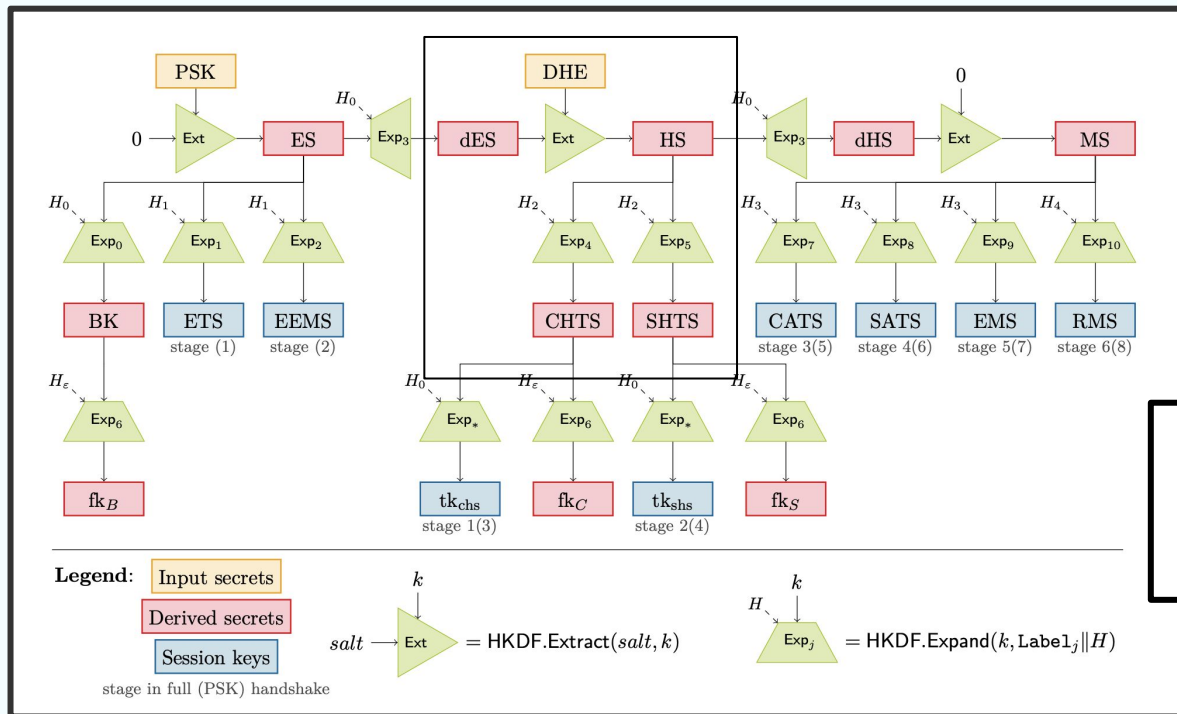
Awesome diagrams from “A Cryptographic Analysis of the TLS 1.3 Handshake Protocol”,  
<https://d1kjiwibowugqa.cloudfront.net/files/research/papers/JC-DFGS21.pdf>

# TLS 1.3: handshake protocol, key schedule



Awesome diagrams from “A Cryptographic Analysis of the TLS 1.3 Handshake Protocol”,  
<https://d1kjiwibowugqa.cloudfront.net/files/research/papers/JC-DFGS21.pdf>

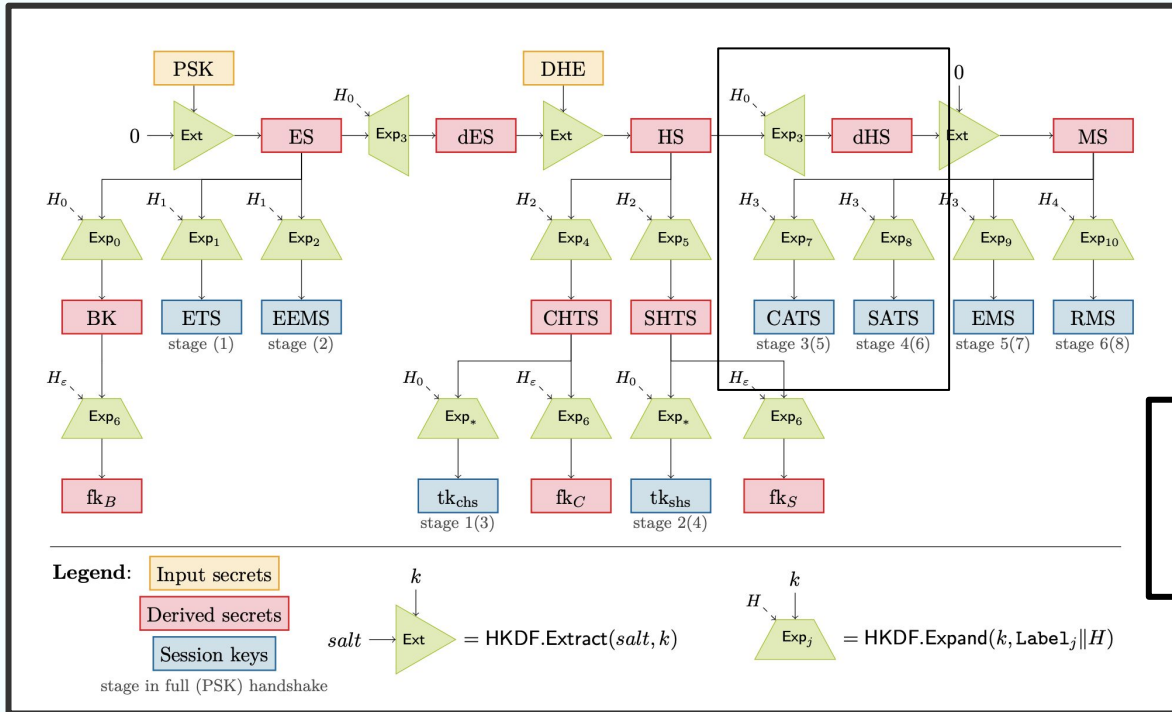
# TLS 1.3: handshake protocol, key schedule



Handshake traffic keys

Awesome diagrams from “A Cryptographic Analysis of the TLS 1.3 Handshake Protocol”,  
<https://d1kjiwibowugqa.cloudfront.net/files/research/papers/JC-DFGS21.pdf>

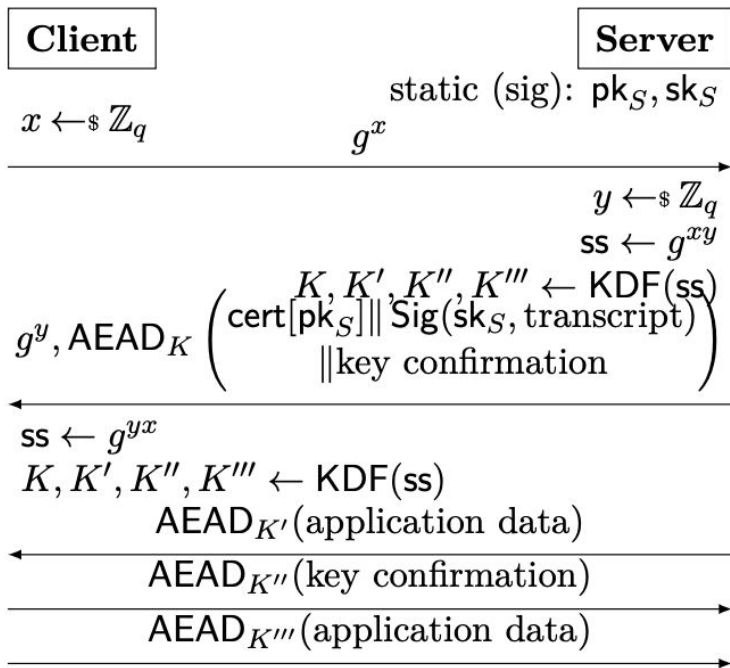
# TLS 1.3: handshake protocol, key schedule



Application layer traffic keys

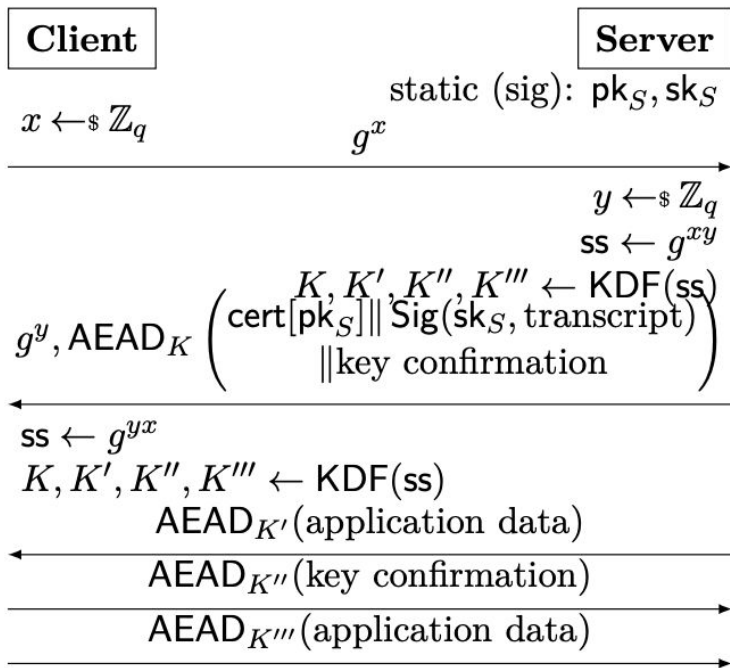
Awesome diagrams from “A Cryptographic Analysis of the TLS 1.3 Handshake Protocol”,  
<https://d1kjiwibowugqa.cloudfront.net/files/research/papers/JC-DFGS21.pdf>

# TLS 1.3: handshake protocol, key schedule



- One *shared (EC)DHE secret* is turned into many independent keys with HKDF (extract, then expand with labels)
- Separate keys per direction:
  - `client_*_traffic_secret` is what the client writes and the server reads
  - `server_*_traffic_secret` is what the server writes and the client reads
  - This blocks reflection attacks, and the two directions never share a key and nonce

# TLS 1.3: handshake protocol, key schedule



- Separate keys per phase:
  - *Handshake traffic keys* come from (EC)DHE plus the transcript up to ServerHello. They protect the rest of the handshake, before anyone is authenticated
  - *Application traffic keys* are derived later, from the transcript up to the server's Finished. They are bound to the authenticated handshake
- Compromise of one does not reveal the other
- Every secret has its own label and its own transcript hash (domain separation), so a key derived for one purpose is never used for another

# TLS 1.3: handshake protocol, key exchange

```

    {EncryptedExtensions}:  $\vec{ext}_S$ 
    {CertificateRequest}*
    {ServerCertificate}:  $pk_S$ 
{ServerCertVfy}:  $SCV \leftarrow \text{SIG.Sign}(sk_S, \text{Label}_{11} \| H_6)$ 
 $fk_S \leftarrow \text{HKDF.Expand}(\text{SHTS}, \text{Label}_6 \| H_6)$ 
    {ServerFinished}:  $SF \leftarrow \text{HMAC}(fk_S, H_7)$ 

abort if  $\text{SIG.Vfy}(pk_S, \text{Label}_{11} \| H_6, SCV) \neq 1$ 
abort if  $SF \neq \text{HMAC}(fk_S, H_7)$ 
     $MS \leftarrow \text{HKDF.Extract}(\text{dHS}, 0)$ 
    accept CATS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_7 \| H_3)$  ..... stage 3
    accept SATS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_8 \| H_3)$  ..... stage 4
    accept EMS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_9 \| H_3)$  ..... stage 5
record layer, AEAD-encrypted with key  $tk_{\text{sapp}}$  (optional)
←-----
{ClientCertificate}*:  $pk_C$ 
{ClientCertVfy}*:  $CCV \leftarrow \text{SIG.Sign}(sk_C, \text{Label}_{12} \| H_8)$ 
 $fk_C \leftarrow \text{HKDF.Expand}(\text{CHTS}, \text{Label}_6 \| H_6)$ 
{ClientFinished}:  $CF \leftarrow \text{HMAC}(fk_C, H_9)$ 

    abort if  $\text{SIG.Vfy}(pk_C, \text{Label}_{12} \| H_8, CCV) \neq 1$ 
    abort if  $CF \neq \text{HMAC}(fk_C, H_9)$ 
    accept RMS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_{10} \| H_4)$  ..... stage 6
record layer, AEAD-encrypted with key  $tk_{\text{capp}}$  →
←-----
record layer, AEAD-encrypted with key  $tk_{\text{sapp}}$  ←-----
```

The **Authentication** and **Integrity** phase:

- Server sends Certificate and CertificateVerify messages, which advertises its authentication public key and a signature with the secret counterpart: signs a hash of the handshake messages up to that point
- It can request the client to optionally authenticate

Awesome diagrams from “A Cryptographic Analysis of the TLS 1.3 Handshake Protocol”,  
<https://d1kjiwibowugqa.cloudfront.net/files/research/papers/JC-DFGS21.pdf>

# TLS 1.3: handshake protocol, key exchange

```

    {EncryptedExtensions}:  $\vec{ext}_S$ 
    {CertificateRequest}*
    {ServerCertificate}:  $pk_S$ 
{ServerCertVfy}:  $SCV \leftarrow \text{SIG.Sign}(sk_S, \text{Label}_{11} \| H_6)$ 
     $fk_S \leftarrow \text{HKDF.Expand}(\text{SHTS}, \text{Label}_6 \| H_\epsilon)$ 
    {ServerFinished}:  $SF \leftarrow \text{HMAC}(fk_S, H_7)$ 

abort if  $\text{SIG.Vfy}(pk_S, \text{Label}_{11} \| H_6, SCV) \neq 1$ 
abort if  $SF \neq \text{HMAC}(fk_S, H_7)$ 
     $MS \leftarrow \text{HKDF.Extract}(\text{dHS}, 0)$ 
    accept CATS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_7 \| H_3)$  ..... stage 3
    accept SATS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_8 \| H_3)$  ..... stage 4
    accept EMS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_9 \| H_3)$  ..... stage 5
record layer, AEAD-encrypted with key  $tk_{sapp}$  (optional)
←-----
{ClientCertificate}*:  $pk_C$ 
{ClientCertVfy}*:  $CCV \leftarrow \text{SIG.Sign}(sk_C, \text{Label}_{12} \| H_8)$ 
     $fk_C \leftarrow \text{HKDF.Expand}(\text{CHTS}, \text{Label}_6 \| H_\epsilon)$ 
    {ClientFinished}:  $CF \leftarrow \text{HMAC}(fk_C, H_9)$ 

    abort if  $\text{SIG.Vfy}(pk_C, \text{Label}_{12} \| H_8, CCV) \neq 1$ 
    abort if  $CF \neq \text{HMAC}(fk_C, H_9)$ 
    accept RMS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_{10} \| H_4)$  ..... stage 6
    record layer, AEAD-encrypted with key  $tk_{capp}$  →
←-----
    record layer, AEAD-encrypted with key  $tk_{sapp}$  ←-----
```

The **Authentication** and **Integrity** phase:

- Server sends `ServerFinished` message:
  - It is a MAC over the entire handshake so far:
    - $\text{Finished} = \text{HMAC}(\text{finished\_key}, \text{Hash}(\text{transcript}))$
    - `finished_key` is derived from the sender's handshake traffic secret (label "finished")
- Provides:
  - **Key confirmation:** only a party that computed the same (EC)DHE secret can produce it
  - **Transcript integrity:** any tampering with the cleartext hellos changes the hash, so downgrade attacks are detected.
  - **Binds identity to keys:** it ties the certificate and its signature to this key exchange and this connection

# TLS 1.3: handshake protocol, key exchange

```

    {EncryptedExtensions}:  $\vec{ext}_S$ 
    {CertificateRequest}*
    {ServerCertificate}:  $pk_S$ 
{ServerCertVfy}:  $SCV \leftarrow \text{SIG.Sign}(sk_S, \text{Label}_{11} || H_6)$ 
     $fk_S \leftarrow \text{HKDF.Expand}(\text{SHTS}, \text{Label}_6 || H_\epsilon)$ 
    {ServerFinished}:  $SF \leftarrow \text{HMAC}(fk_S, H_7)$ 

abort if  $\text{SIG.Vfy}(pk_S, \text{Label}_{11} || H_6, SCV) \neq 1$ 
abort if  $SF \neq \text{HMAC}(fk_S, H_7)$ 
     $MS \leftarrow \text{HKDF.Extract}(\text{dHS}, 0)$ 
    accept CATS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_7 || H_3)$  ..... stage 3
    accept SATS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_8 || H_3)$  ..... stage 4
    accept EMS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_9 || H_3)$  ..... stage 5
record layer, AEAD-encrypted with key  $tk_{sapp}$  (optional)
←-----
{ClientCertificate}*:  $pk_C$ 
{ClientCertVfy}*:  $CCV \leftarrow \text{SIG.Sign}(sk_C, \text{Label}_{12} || H_8)$ 
     $fk_C \leftarrow \text{HKDF.Expand}(\text{CHTS}, \text{Label}_6 || H_\epsilon)$ 
    {ClientFinished}:  $CF \leftarrow \text{HMAC}(fk_C, H_9)$ 

    abort if  $\text{SIG.Vfy}(pk_C, \text{Label}_{12} || H_8, CCV) \neq 1$ 
    abort if  $CF \neq \text{HMAC}(fk_C, H_9)$ 
    accept RMS  $\leftarrow \text{HKDF.Expand}(MS, \text{Label}_{10} || H_4)$  ..... stage 6
    record layer, AEAD-encrypted with key  $tk_{capp}$  →
←-----
    record layer, AEAD-encrypted with key  $tk_{sapp}$  ←-----

```

The Authentication and Integrity phase:

- The server can now send any application data!
- But, they still need to see the:
  - ClientFinished message
    - Confirmation that the client derived the same keys
    - Confirmation that both sides saw the same transcript, which detects downgrade from the server's side
    - Client authentication, if requested

## TLS 1.3: a bit on signatures and the PKI

- **Authentication:** Assurance that you are talking with the correct party
  - Party signs with the private key
  - Peer (or anyone) verifies the signature with the public key
- TLS 1.3 allows for: server-only authentication or mutual (both server and client) authentication
  - Need to verify the used public key belongs to the correct party and was-issued/is-maintained correctly
- However, authentication does not end there

## TLS 1.3: a bit on signatures and the PKI

- The amount of signatures in the handshake are many. It could be:
  - two signatures in the certificate chain
  - **one handshake signature**
  - one Online Certificate Status Protocol (OCSP) staple for revocation state
  - two SCTs used for certificate transparency

## TLS 1.3: a bit on signatures and the PKI

- A set of parties
  - **Certificate Chain:** root Certificate signed by the Certificate Authority (CA),  $n$  number of intermediate certificates, and an end-entity (leaf) certificate
  - Authorities are extremely careful
- Each certificate in the chain is signed by the entity identified by the next certificate in the chain (this could be malformed as intermediate deprecated certificates can be present, or misordered)
- “Offline” signing: it does not happen at the handshake phase
- Checks the signature of end-entity certificate, and keeps checking certificates until the root certificate, stored in a trust store (OS or browser)

## TLS 1.3: a bit on signatures and the PKI

- Intermediate certificates:
  - Have to be included in the certificate chain
  - They can be reduced or browsers can cache them:  
<https://datatracker.ietf.org/doc/html/rfc7924>

## TLS 1.3: a bit on signatures and the PKI

- Certificate Transparency:
  - Issued certificates are placed in public logs
  - Issue a *signed certificate timestamp (SCT)*: a promise to include the certificate in the log within some delay
  - SCTs are presented with the certificate to the client: check that the SCTs are valid and other policies. Check inclusion in the log
    - This has a privacy violation
  - Designated auditors and monitors check that logs are append-only, globally consistent and that logs do not contain any inconsistencies or mis-issued certificates

## TLS 1.3: a bit on signatures and the PKI

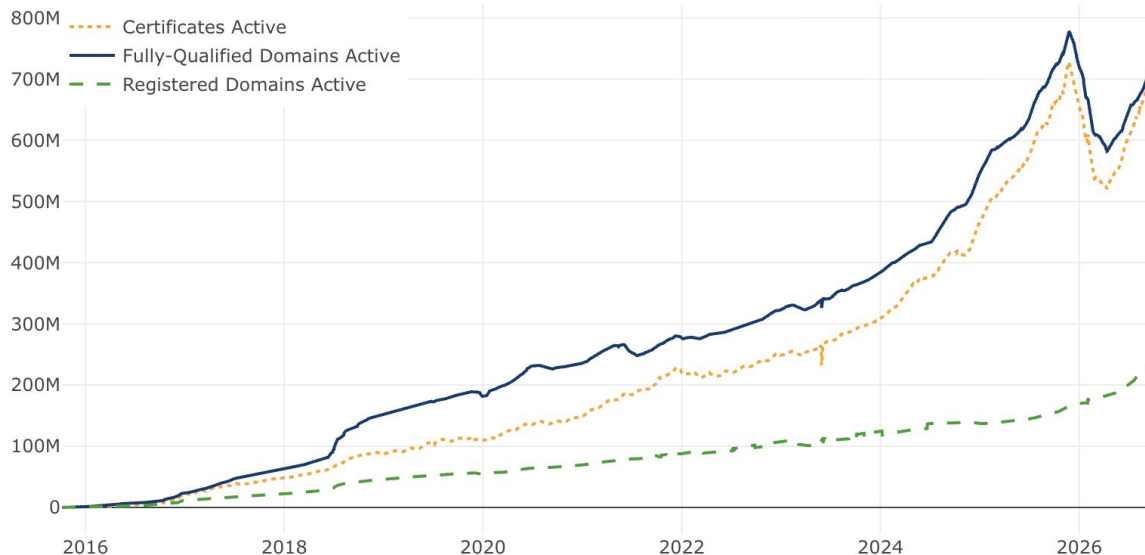
- Certificate Revocation:
  - Online Certificate Status Protocol (OCSP):
    - Privacy violation when checking with the log directly
  - OCSP Stapling: the holder of the certificate staples the check
  - CRL and CRLite
    - Huge databases

## TLS 1.3: a bit on the unpractical side...

- Many problems:
  - Too many signatures to verify
  - Few signatures to create
  - Many public keys to transmit
  - Several privacy violations
  - Two databases to append to, and to check to:
    - Several privacy violations
  - A pain to handle, even with Let's Encrypt..

# TLS 1.3: a bit on the unpractical side...

## Let's Encrypt Usage

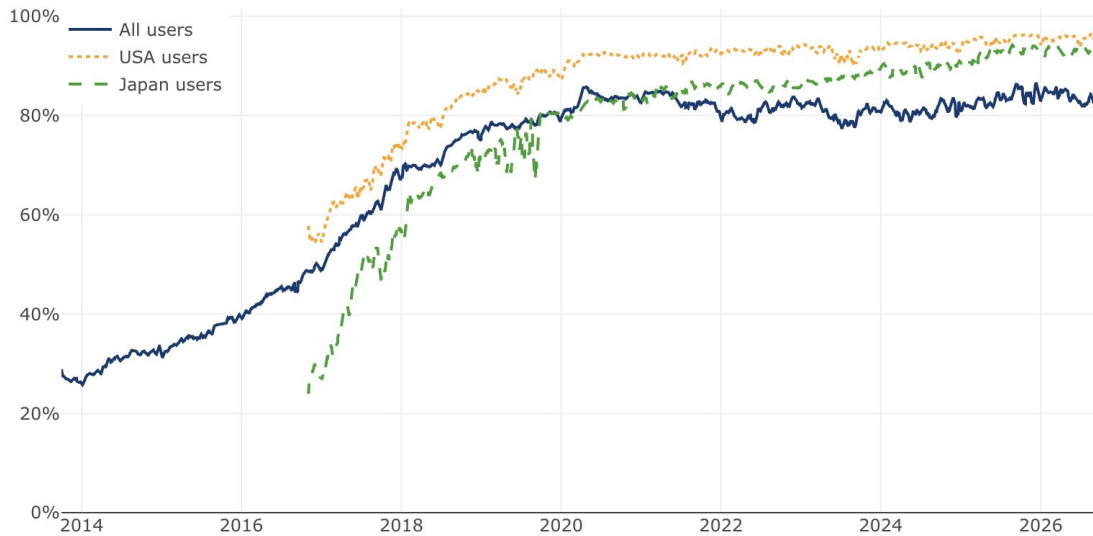


From: <https://letsencrypt.org/stats/>

# TLS 1.3: a bit on the unpractical side...

## Percentage of Web Pages Loaded by Firefox Using HTTPS

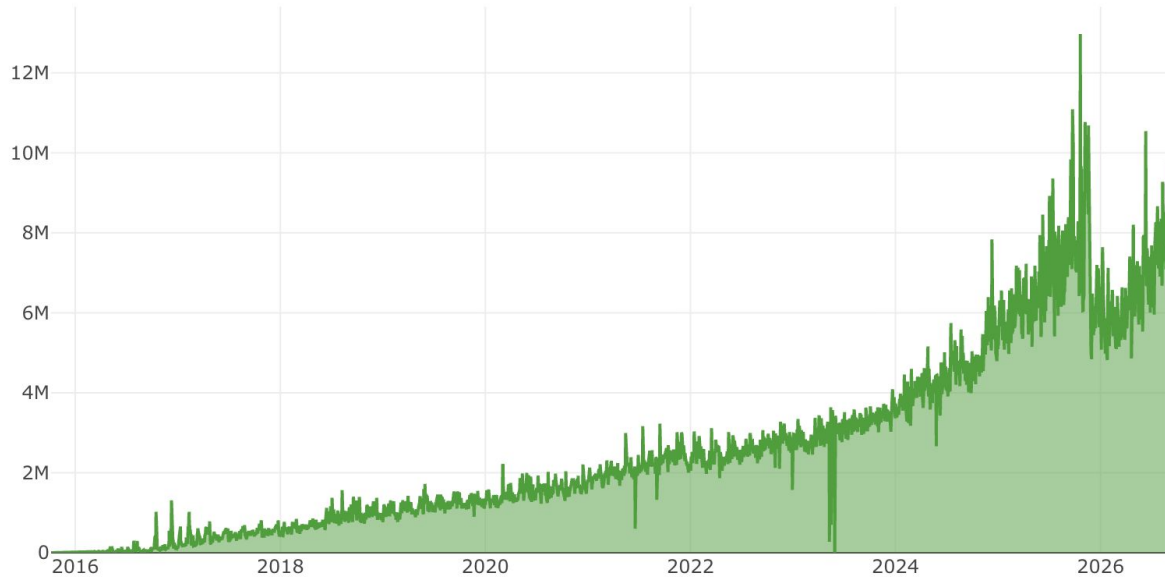
(14-day moving average, source: [Firefox Telemetry](#))



From: <https://letsencrypt.org/stats/>

# TLS 1.3: a bit on the unpractical side...

## Let's Encrypt Certificates Issued Per Day



From: <https://letsencrypt.org/stats/>

# Nevertheless..

TLS 1.3 provides a lot of security assurances:

- Confidentiality: as much as possible (except for sending SNI in the clear, and the `Hello`s messages)
  - Forward secrecy per session
- Authentication
- Integrity

# What about the record layer?

TLS 1.3 only accepts *AEAD algorithms* (AES-GCM, ChaCha20-Poly1305, AES-CCM)

- Encryption and authentication happen in one step, and integrity is checked before any plaintext is analysed
- TLS  $\leq$  1.2 CBC suites used *MAC-then-pad-then-encrypt*, which allowed padding oracles (*Lucky13*, *POODLE*)
- Any failure produces a single `bad_record_mac` alert and the connection closes. The error gives an attacker nothing to distinguish, so there is no oracle
- The nonce is implicit: `write_iv XOR 64-bit sequence number`
  - It is never sent on the wire, so it can't be manipulated or accidentally reused
  - The sequence number is per direction and resets on every key change (`KeyUpdate`)
  - Replayed, reordered or dropped records fail authentication
- The record header is authenticated as additional data. The real content type is inside the ciphertext:
  - Handshake messages, alerts and application data all look like `application_data` on the wire
  - Optional zero padding can hide record lengths
  - Removed entirely: compression (CRIME), RC4, CBC, 3DES, renegotiation

# What about the record layer?

---

**Algorithm 3** AES-GCM.Enc( $k, IV; M, A$ ).

---

**Input:** key  $k$ , nonce  $IV$ , associated data  $A$ , message  $M$ .

**Output:** ciphertext  $C$ , tag  $\tau$ .

**Precondition:**  $IV$  has not been used with  $k$  before.

---

*// Encrypt (Confidentiality)*

1: decompose  $M$  as  $M_1 \parallel \dots \parallel M_n$  ▷ 16-byte blocks

2: **for all**  $i \in [n]$  **do**

3:      $C_i \leftarrow \text{AES.Enc}(k, IV \parallel \langle i \rangle) \oplus M_i$  ▷ CTR mode

4: **end for**

5:  $C \leftarrow C_1 \parallel \dots \parallel C_n$

*// MAC (Integrity)*

6:  $H \leftarrow \text{AES.Enc}(k, 0^{128})$  ▷ GHASH key

7:  $X \leftarrow A \parallel C \parallel \text{len}(A) \parallel \text{len}(C)$

*//  $P_X(H) = \bigoplus_j X_j \cdot H^{n'-j+1}$  over  $\mathbb{F}_{2^{128}}$*

8:  $\tau \leftarrow P_X(\{H^i\}_{i \in [n']}) \oplus \text{AES.Enc}(k, IV \parallel \langle 0 \rangle)$

9: **return**  $(C, \tau)$

---

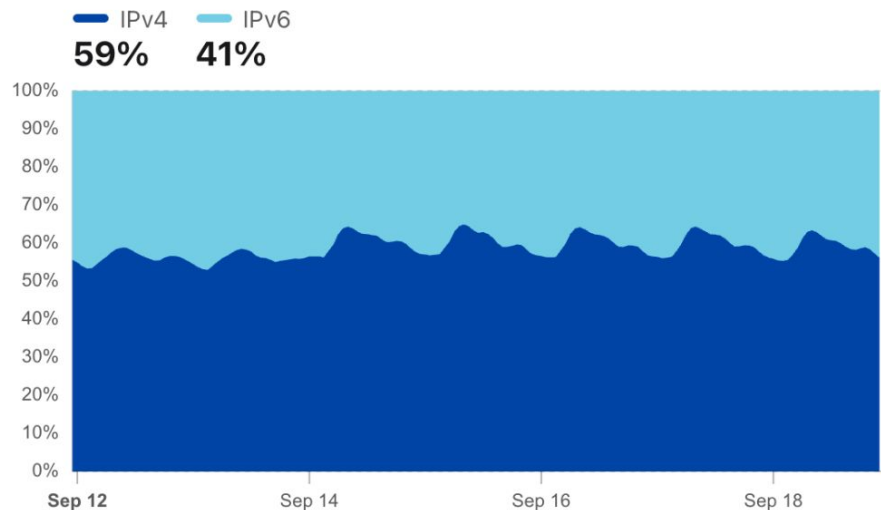
## So, is it practical?

- From a complexity level... not really
- But it does give the best assurances:
  - It was not always perfect, there was a huge effort from academia and industry to verify it:
    - One of the few protocols that is formally verified end-to-end
- It is easy to implement? Not really, but we have specialised knowledge of it
- Does it work over anything? yes.

# So, is it practical?

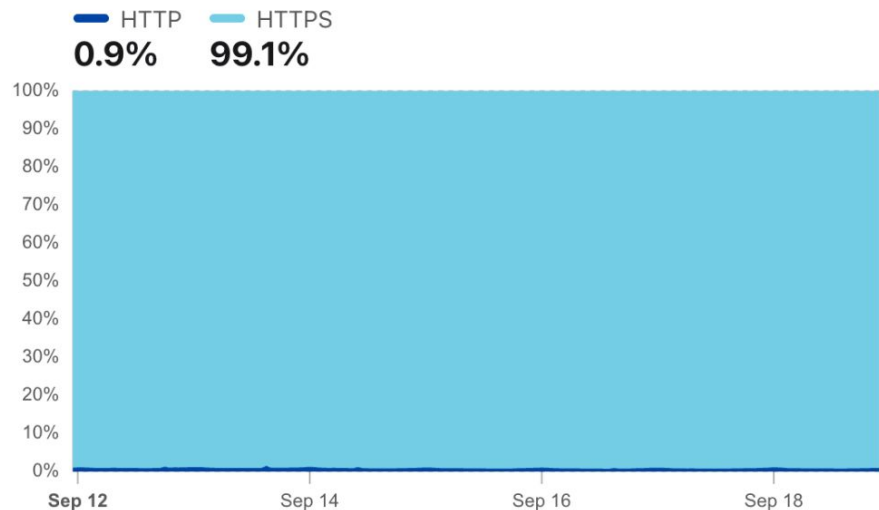
## IPv4 vs. IPv6

Distribution of HTTP requests by IP version (?) 🔗 ⋮



## HTTP vs. HTTPS

Distribution of HTTP vs. HTTPS requests (?) 🔗 ⋮

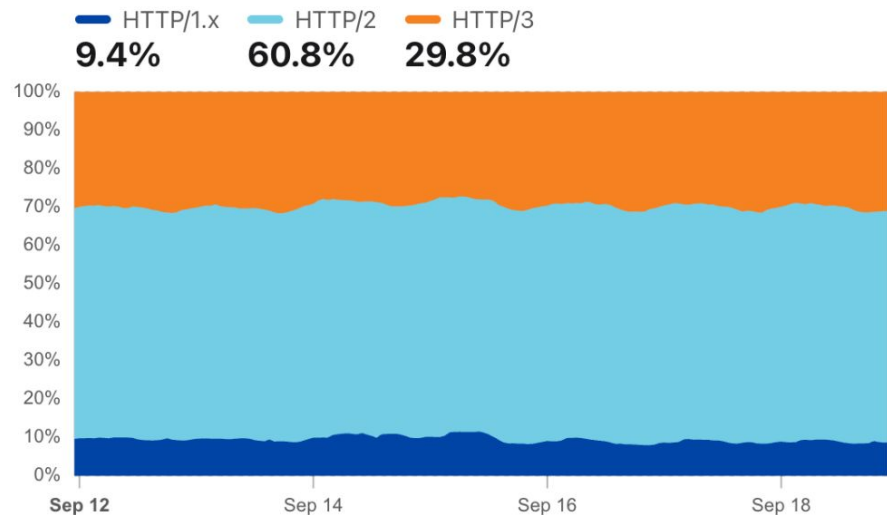


From: <https://radar.cloudflare.com/>

# So, is it practical?

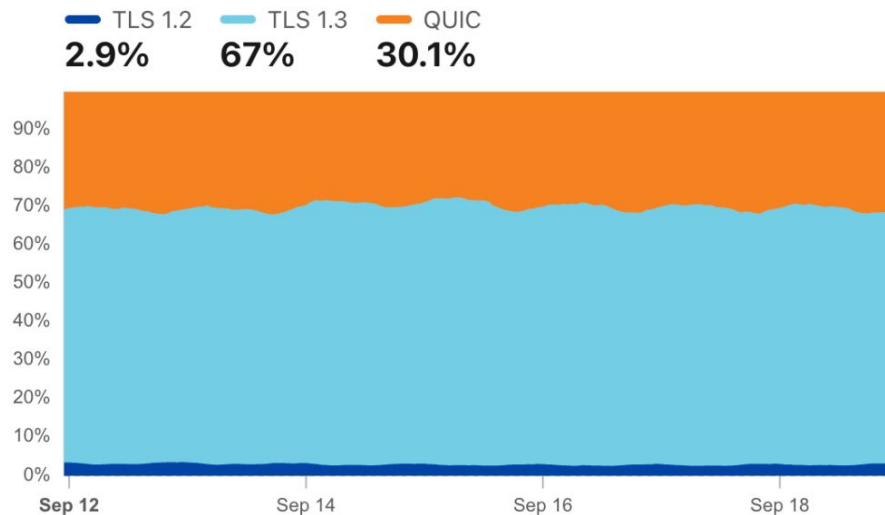
## HTTP/1.x vs. HTTP/2 vs. HTTP/3

Distribution of requests by HTTP version ? 🔗 ⋮



## TLS 1.2 vs. TLS 1.3 vs. QUIC

Distribution of secure HTTP requests by protocol ? 🔗 ⋮



From: <https://radar.cloudflare.com/>

## Why we trust it..

- To a degree: because a lot of people have looked into it: academics, industry and standarization bodies
- Standarization does show the interest of groups on cryptography

# Why we trust it..

From: TLS <tls-bounces@ietf.org> on behalf of BITS Security <BITSSecurity@fsroundtable.org>

Sent: Thursday, September 22, 2016 10:19:48 AM

To: [tls@ietf.org](mailto:tls@ietf.org)

Subject: [TLS] Industry Concerns about TLS 1.3

To: IETF TLS 1.3 Working Group Members

My name is Andrew Kennedy and I work at BITS, the technology policy division of the Financial Services Roundtable (<http://www.fsroundtable.org/bits>) My organization represents approximately 100 of the top 150 US-based financial services companies including banks, insurance, consumer finance, and asset management firms.

I manage the Technology Cybersecurity Program, a CISO-driven forum to investigate emerging technologies; integrate capabilities into member operations; and advocate member, sector, cross-sector, and private-public collaboration.

While I am aware and on the whole supportive of the significant contributions to internet security this important working group has made in the last few years I recently learned of a proposed change that would affect many of my organization's member institutions: the deprecation of RSA key exchange.

Deprecation of the RSA key exchange in TLS 1.3 will cause significant problems for financial institutions, almost all of whom are running TLS internally and have significant, security-critical investments in out-of-band TLS decryption.

Like many enterprises, financial institutions depend upon the ability to decrypt TLS traffic to implement data loss protection, intrusion detection and prevention, malware detection, packet capture and analysis, and DDoS mitigation. Unlike some other businesses, financial institutions also rely upon TLS traffic decryption to implement fraud monitoring and surveillance of supervised employees. The products which support these capabilities will need to be replaced or substantially redesigned at significant cost and loss of scalability to continue to support the functionality financial institutions and their regulators require.

# Why we trust it..



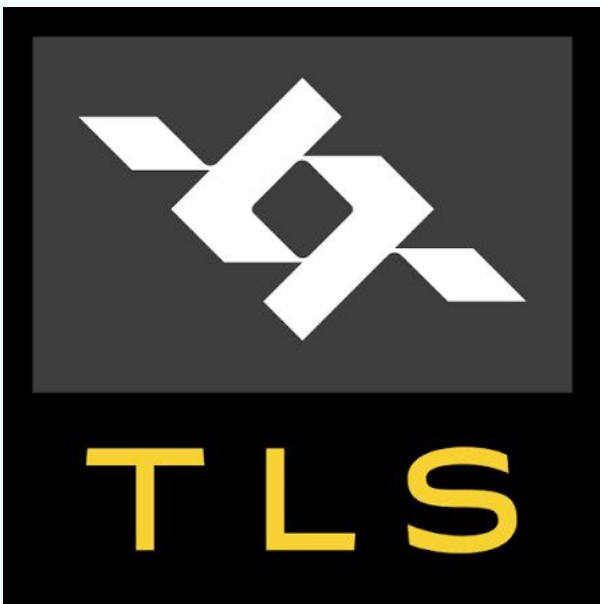
The screenshot shows a web browser window with the address bar displaying `mailarchive.ietf.org/arch/msg/tls/x4An2ELcfHWkulfWioXdTdFiDHo/`. The page header features the IETF logo and the text "Mail Archive". Below the header, there are navigation links: "< Date >" and "< Thread >". The main content area displays the subject line "Re: [TLS] draft-green-tls-static-dh-in-tls13-01" and the sender information "Jacob Hoffman-Andrews <jsha@eff.org> | Sat, 08 July 2017 22:08 UTC | [Show header](#)". The body of the email contains two paragraphs of text.

Re: [TLS] draft-green-tls-static-dh-in-tls13-01

Jacob Hoffman-Andrews <jsha@eff.org> | Sat, 08 July 2017 22:08 UTC | [Show header](#)

With all respect to the authors for a well-written paper, this draft should not be adopted by TLS WG, because it would weaken the security of TLS as deployed in practice.

If the TLS WG were to adopt this draft, one of the likely outcomes would be implementation in a variety of popular TLS libraries, like OpenSSL and SChannel. Unfortunately, including interception capability in off-the-shelf software is like keeping your plutonium next to your potato chips; something is bound to go wrong. For instance, performance advocates may suggest turning on the interception capability in order to save cycles. If such advice is broadly adopted, we will effectively have reintroduced non-forward secret sessions to TLS. In TLS 1.3 there are no NULL cipher suites. Why should there be a commonly implemented mode that breaks forward secrecy?



**THANK YOU!**

@claucece

<https://sofiaceli.com/>